

Seminar 8

Introducere în Programarea Liniară

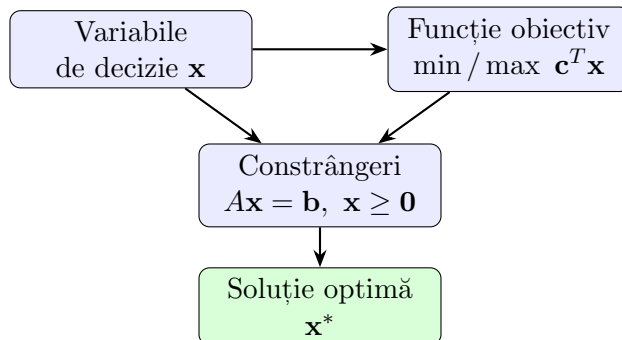
Asist. univ. dr. Mihai Tudor

21 aprilie 2026

1 Ce este Programarea Liniară?

Programarea liniară (PL) este ramura optimizării matematice care studiază minimizarea sau maximizarea unei funcții liniare, supusă unui sistem de inegalități și/sau egalități liniare.

Apare în mod natural ori de câte ori resursele sunt limitate și trebuie alocate eficient: planificarea producției, logistică, alimentație, finanțe, transport.



2 Formulare matematică

Definiție 2.1. O problemă de programare liniară (PL) în formă standard este:

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} \quad & \mathbf{c}^T \mathbf{x} \\ \text{s.t.} \quad & A\mathbf{x} = \mathbf{b}, \quad \mathbf{x} \geq \mathbf{0} \end{aligned}$$

unde:

- $\mathbf{x} = (x_1, \dots, x_n)^T \in \mathbb{R}^n$ — **vectorul variabilelor de decizie**
- $\mathbf{c} = (c_1, \dots, c_n)^T \in \mathbb{R}^n$ — **vectorul costurilor** (coeficienții funcției obiectiv)
- $A \in \mathbb{R}^{m \times n}$ — **matricea constrângerilor**
- $\mathbf{b} \in \mathbb{R}^m$ — **vectorul de resurse**

Observație 2.1. Reducerea la forma standard. Orice problemă PL se poate aduce la forma de mai sus:

Formă inițială	Echivalent în forma standard
$\max \mathbf{c}^T \mathbf{x}$	$\min (-\mathbf{c})^T \mathbf{x}$
$a_i^T \mathbf{x} \geq b_i$	$(-a_i)^T \mathbf{x} \leq -b_i$
$a_i^T \mathbf{x} = b_i$	$a_i^T \mathbf{x} \leq b_i$ și $(-a_i)^T \mathbf{x} \leq -b_i$
x_j variabilă liberă	$x_j = x_j^+ - x_j^-, \quad x_j^+, x_j^- \geq 0$

Definiție 2.2. Fie o problemă PL. Definim:

- **Mulțimea acceptabilă** (regiunea admisibilă): $\mathcal{F} = \{\mathbf{x} \geq \mathbf{0} \mid A\mathbf{x} \leq \mathbf{b}\}$
- O soluție $\mathbf{x} \in \mathcal{F}$ se numește **acceptabilă**.

- $\mathbf{x}^* \in \mathcal{F}$ cu $\mathbf{c}^T \mathbf{x}^* \leq \mathbf{c}^T \mathbf{x}$ pentru orice $\mathbf{x} \in \mathcal{F}$ se numește **soluție optimă**.
- Valoarea $z^* = \mathbf{c}^T \mathbf{x}^*$ se numește **valoarea optimă**.

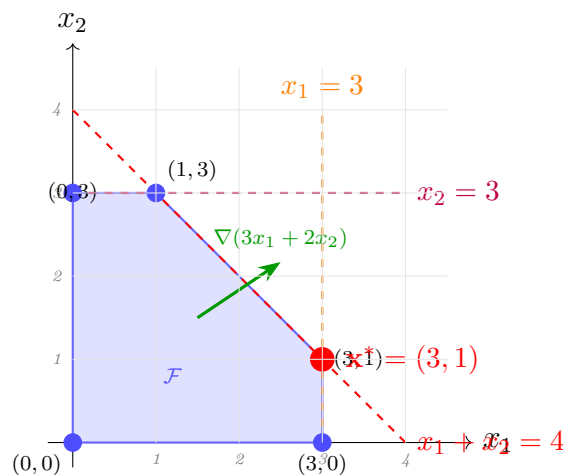
Teoremă 2.1 (Teorema fundamentală a PL). Dacă mulțimea fezabilă $\mathcal{F}$ este nevidă și mărginită, atunci problema PL admite cel puțin o soluție optimă, iar aceasta se atinge într-un **vârf** (vertex) al mulțimii $\mathcal{F}$.

Observație 2.2. Mulțimea fezabilă $\mathcal{F}$ a unei probleme PL este un **poliedru convex** (intersecție de semispații). Vârfurile sale se numesc **soluții de bază acceptabile** și sunt punctele în care n constrângeri sunt active (satisfăcute cu egalitate). Algoritmul Simplex explorează vârfurile poliedrului în mod sistematic.

3 Ilustrare geometrică în 2 dimensiuni

Exemplu 3.1. Problemă de referință în 2D:

$$\begin{aligned} \max \quad & 3x_1 + 2x_2 \\ \text{s.t.} \quad & x_1 + x_2 \leq 4, \quad x_1 \leq 3, \quad x_2 \leq 3, \quad x_1, x_2 \geq 0 \end{aligned}$$



Evaluarea funcției obiectiv în vârfuri:

Vârf	$3x_1 + 2x_2$	
(0, 0)	0	
(3, 0)	9	
(3, 1)	11	← OPTIM
(1, 3)	9	
(0, 3)	6	

Soluția optimă este $\mathbf{x}^* = (3, 1)$ cu valoarea optimă $z^* = 11$.

4 Problema dietei

Enunț

Un nutriționist dorește să planifice o dietă zilnică minimă ca **cost**, care să satisfacă necesarul de calorii, proteine și calciu. Alimentele disponibile sunt **Pâine** (x_1), **Lapte** (x_2) și **Ouă** (x_3), măsurate în porții.

Nutrient	Pâine	Lapte	Ouă	Necesar minim
Calorii (kcal/porție)	200	150	100	≥ 1800
Proteine (g/porție)	4	8	12	≥ 60
Calciu (mg/porție)	20	200	30	≥ 800
Cost (lei/porție)	1.0	1.5	2.0	<i>de minimizat</i>

Formulare matematică

Variabile: x_1 = porții de pâine, x_2 = porții de lapte, x_3 = porții de ouă.

Funcție obiectiv:

$$\min \quad z = x_1 + 1.5x_2 + 2x_3$$

Constrângeri:

$$200x_1 + 150x_2 + 100x_3 \geq 1800 \quad (\text{calorii})$$

$$4x_1 + 8x_2 + 12x_3 \geq 60 \quad (\text{proteine})$$

$$20x_1 + 200x_2 + 30x_3 \geq 800 \quad (\text{calciu})$$

$$x_1, x_2, x_3 \geq 0$$

Forma matriceală: $\min \mathbf{c}^T \mathbf{x}$ cu $A\mathbf{x} \geq \mathbf{b}$, $\mathbf{x} \geq \mathbf{0}$, unde:

$$\mathbf{c} = \begin{pmatrix} 1 \\ 1.5 \\ 2 \end{pmatrix}, \quad A = \begin{pmatrix} 200 & 150 & 100 \\ 4 & 8 & 12 \\ 20 & 200 & 30 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1800 \\ 60 \\ 800 \end{pmatrix}$$

Implementare Python

```

1 from scipy.optimize import linprog
2 import numpy as np
3
4 # -----
5 # Problema dietei
6 # Variabile: x1=Paine, x2=Lapte, x3=Oua (portii/zi)
7 # -----
8
9 # Coeficientii functiei obiectiv (costuri lei/portie)
10 c = [1.0, 1.5, 2.0]
11
12 # Matricea constrangerilor (>= se transforma in <= prin negare)
13 # 200x1+150x2+100x3 >= 1800 => -200x1-150x2-100x3 <= -1800

```

```

14 A_ub = [
15     [-200, -150, -100], # calorii (negat)
16     [-4, -8, -12], # proteine (negat)
17     [-20, -200, -30] # calciu (negat)
18 ]
19 b_ub = [-1800, -60, -800] # membre drepte (negate)
20
21 # Marginile variabilelor:  $x_i \geq 0$ 
22 bounds = [(0, None), (0, None), (0, None)]
23
24 # Rezolvare
25 rezultat = linprog(c, A_ub=A_ub, b_ub=b_ub,
26                    bounds=bounds, method='highs')
27
28 # Afisare rezultate
29 print("=== PROBLEMA DIETEI ===")
30 print(f"Status: {rezultat.message}")
31 print(f"Portii paine: x1 = {rezultat.x[0]:.4f}")
32 print(f"Portii lapte: x2 = {rezultat.x[1]:.4f}")
33 print(f"Portii oua: x3 = {rezultat.x[2]:.4f}")
34 print(f"Cost minim: z = {rezultat.fun:.4f} lei")
35
36 # Verificarea constrangerilor
37 x = rezultat.x
38 print("\nVerificarea constrangerilor:")
39 print(f" Calorii: {200*x[0]+150*x[1]+100*x[2]:.1f} >= 1800")
40 print(f" Proteine: {4*x[0]+8*x[1]+12*x[2]:.1f} >= 60")
41 print(f" Calciu: {20*x[0]+200*x[1]+30*x[2]:.1f} >= 800")
42
43 # Output:
44 # Status: Optimization terminated successfully.
45 # Portii paine: x1 = 6.1657
46 # Portii lapte: x2 = 3.2686
47 # Portii oua: x3 = 0.7657
48 # Cost minim: z = 12.6000 lei
49 # Calorii: 1800.0 >= 1800
50 # Proteine: 60.0 >= 60
51 # Calciu: 800.0 >= 800

```

Observație 4.1. Constrângerile de tip $\geq$ se transformă în $\leq$ prin înmulțire cu -1 . Aceasta este transformarea standard pentru `scipy.optimize.linprog`, care acceptă numai constrângeri $\leq$.

5 Problema transportului

Enunț

Două depozite (S_1, S_2) aprovizionează trei magazine (D_1, D_2, D_3). Se cunoaște:

- **Oferta** depozitelor: $S_1 = 300$ unități, $S_2 = 400$ unități.

- **Cererea** magazinelor: $D_1 = 200$, $D_2 = 250$, $D_3 = 250$ unități.
- **Costul** de transport (lei/unitate):

	D_1	D_2	D_3	Ofertă
S_1	2	3	1	300
S_2	5	4	8	400
Cerere	200	250	250	700

Oferta totală = $300 + 400 = 700$ = cererea totală. Problema este **echilibrată**.

Formulare matematică

Variabile: x_{ij} = cantitatea transportată de la S_i la D_j , cu $i \in \{1, 2\}$, $j \in \{1, 2, 3\}$.

Funcție obiectiv:

$$\min \quad z = 2x_{11} + 3x_{12} + x_{13} + 5x_{21} + 4x_{22} + 8x_{23}$$

Constrângeri de ofertă (cantitatea expediată = oferta disponibilă):

$$x_{11} + x_{12} + x_{13} = 300 \quad (S_1)$$

$$x_{21} + x_{22} + x_{23} = 400 \quad (S_2)$$

Constrângeri de cerere (cantitatea primită = cererea):

$$x_{11} + x_{21} = 200 \quad (D_1)$$

$$x_{12} + x_{22} = 250 \quad (D_2)$$

$$x_{13} + x_{23} = 250 \quad (D_3)$$

$$x_{ij} \geq 0 \quad \text{pentru toți } i, j$$

Forma matriceală cu $\mathbf{x} = (x_{11}, x_{12}, x_{13}, x_{21}, x_{22}, x_{23})^T$:

$$\mathbf{c}^T = (2, 3, 1, 5, 4, 8)$$

$$A_{eq} = \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \end{pmatrix}, \quad \mathbf{b}_{eq} = \begin{pmatrix} 300 \\ 400 \\ 200 \\ 250 \\ 250 \end{pmatrix}$$

Implementare Python

```

1  from scipy.optimize import linprog
2
3  # -----
4  # Problema transportului
5  # x = [x11, x12, x13, x21, x22, x23]
6  # -----
7
8  # Costurile de transport
9  c = [2, 3, 1, 5, 4, 8]
10
11 # Constrangeri de egalitate: oferta si cerere
12 A_eq = [
13     [1, 1, 1, 0, 0, 0], # oferta S1 = 300
14     [0, 0, 0, 1, 1, 1], # oferta S2 = 400
15     [1, 0, 0, 1, 0, 0], # cerere D1 = 200
16     [0, 1, 0, 0, 1, 0], # cerere D2 = 250
17     [0, 0, 1, 0, 0, 1], # cerere D3 = 250
18 ]
19 b_eq = [300, 400, 200, 250, 250]
20
21 bounds = [(0, None)] * 6 # xij ≥ 0
22
23 # Rezolvare
24 rezultat = linprog(c, A_eq=A_eq, b_eq=b_eq,
25                    bounds=bounds, method='highs')
26
27 x = rezultat.x
28 print("=== PROBLEMA TRANSPORTULUI ===")
29 print(f"Status: {rezultat.message}")
30 print(f"\nPlan de transport optim:")
31 print(f"  S1 -> D1: {x[0]:.0f} unitati")
32 print(f"  S1 -> D2: {x[1]:.0f} unitati")
33 print(f"  S1 -> D3: {x[2]:.0f} unitati")
34 print(f"  S2 -> D1: {x[3]:.0f} unitati")
35 print(f"  S2 -> D2: {x[4]:.0f} unitati")
36 print(f"  S2 -> D3: {x[5]:.0f} unitati")
37 print(f"\nCost minim total: {rezultat.fun:.0f} lei")
38
39 # Verificare
40 print("\nVerificare:")
41 print(f"  Oferta S1 utilizata: {x[0]+x[1]+x[2]:.0f} = 300")
42 print(f"  Oferta S2 utilizata: {x[3]+x[4]+x[5]:.0f} = 400")
43 print(f"  Cerere D1 acoperita: {x[0]+x[3]:.0f} = 200")
44 print(f"  Cerere D2 acoperita: {x[1]+x[4]:.0f} = 250")
45 print(f"  Cerere D3 acoperita: {x[2]+x[5]:.0f} = 250")
46
47 # Output:
48 # Plan de transport optim:
49 #   S1 -> D1: 50 unitati

```

```

50 # S1 -> D2: 0 unitati
51 # S1 -> D3: 250 unitati
52 # S2 -> D1: 150 unitati
53 # S2 -> D2: 250 unitati
54 # S2 -> D3: 0 unitati
55 # Cost minim total: 2100 lei

```

6 Problema silozului

Enunț

Un administrator de siloz dorește să **maximizeze profitul** din depozitarea cerealelor. Dispune de un siloz cu capacitate de **1000 tone** și poate depozita trei tipuri de cereale: **Grâu** (x_1), **Porumb** (x_2), **Orz** (x_3).

	Grâu (x_1)	Porumb (x_2)	Orz (x_3)
Profit (lei/tonă)	120	90	80
Cantitate maximă (tone)	500	600	400

Condiție suplimentară: cantitatea totală de grâu și porumb trebuie să fie de cel puțin 400 tone (contracte de livrare asigurate).

Formulare matematică

Funcție obiectiv:

$$\max \quad z = 120x_1 + 90x_2 + 80x_3$$

Constrângeri:

$$x_1 + x_2 + x_3 \leq 1000 \quad (\text{capacitate siloz})$$

$$x_1 \leq 500 \quad (\text{contract grâu})$$

$$x_2 \leq 600 \quad (\text{contract porumb})$$

$$x_3 \leq 400 \quad (\text{contract orz})$$

$$x_1 + x_2 \geq 400 \quad (\text{min. cereale principale})$$

$$x_1, x_2, x_3 \geq 0$$

Transformare pentru linprog: maximizare $\Leftrightarrow$ minimizare a negativului:

$$\min \quad (-120x_1 - 90x_2 - 80x_3)$$

Implementare Python

```

1 from scipy.optimize import linprog
2
3 # -----
4 # Problema silozului
5 # x = [x1 (grau), x2 (porumb), x3 (orz)]
6 # -----
7
8 # Maximizare profit => minimizare (-profit)
9 c = [-120, -90, -80]
10
11 # Constrangeri de tip <=
12 A_ub = [
13     [ 1,  1,  1], # capacitate totala <= 1000
14     [ 1,  0,  0], # grau <= 500
15     [ 0,  1,  0], # porumb <= 600
16     [ 0,  0,  1], # orz <= 400
17     [-1, -1,  0], # -(grau+porumb) <= -400  <=> grau+porumb >=
        400
18 ]
19 b_ub = [1000, 500, 600, 400, -400]
20
21 bounds = [(0, None), (0, None), (0, None)]
22
23 # Rezolva
24 rezultat = linprog(c, A_ub=A_ub, b_ub=b_ub,
25                    bounds=bounds, method='highs')
26
27 x = rezultat.x
28 print("=== PROBLEMA SILOZULUI ===")
29 print(f"Status: {rezultat.message}")
30 print(f"Grau:    x1 = {x[0]:.2f} tone")
31 print(f"Porumb: x2 = {x[1]:.2f} tone")
32 print(f"Orz:     x3 = {x[2]:.2f} tone")
33 # Profitul maxim = -valoarea minima obtinuta
34 print(f"Profit maxim: {-rezultat.fun:.2f} lei")
35
36 print("\nVerificarea constrangerilor:")
37 print(f"  Capacitate:    {x[0]+x[1]+x[2]:.1f} <= 1000")
38 print(f"  Grau maxim:     {x[0]:.1f} <= 500")
39 print(f"  Porumb maxim:    {x[1]:.1f} <= 600")
40 print(f"  Orz maxim:       {x[2]:.1f} <= 400")
41 print(f"  Grau+Porumb:     {x[0]+x[1]:.1f} >= 400")
42
43 # Output:
44 # Grau:    x1 = 500.00 tone
45 # Porumb:  x2 = 500.00 tone
46 # Orz:     x3 = 0.00 tone
47 # Profit maxim: 105000.00 lei

```

Observație 6.1. Interpretarea soluției: profitul maxim de 105.000 lei se obține prin depozitarea a 500 tone grâu (maxim permis prin contract) și 500 tone porumb, fără orz. Orzul nu este depozitat deoarece are cel mai mic profit per tonă (80 lei/tonă vs. 90 și 120) și silozul este deja plin. Constrângerea de minimum 400 tone grâu+porumb este satisfăcută cu mult ($1000 \geq 400$).

7 Implementare unificată

```

1 from scipy.optimize import linprog
2
3 def rezolva_pl(c, A_ub=None, b_ub=None, A_eq=None, b_eq=None,
4               bounds=None, tip='min', nume_variabile=None):
5     """
6     Rezolva o problema de programare liniara si afiseaza
7     rezultatul.
8
9     Parametri:
10    c      -- coeficientii functiei obiectiv
11    A_ub   -- matricea constrangerilor de tip <=
12    b_ub   -- vectorul drept al constrangerilor <=
13    A_eq   -- matricea constrangerilor de egalitate
14    b_eq   -- vectorul drept al egalitatilor
15    bounds-- marginile variabilelor [(lb, ub), ...]
16    tip    -- 'min' sau 'max'
17    nume_variabile -- lista cu numele variabilelor
18    """
19    n = len(c)
20    if bounds is None:
21        bounds = [(0, None)] * n
22    if nume_variabile is None:
23        nume_variabile = [f'x{i+1}' for i in range(n)]
24
25    # Daca maximizam, negam coeficientii
26    c_optim = [-ci for ci in c] if tip == 'max' else list(c)
27
28    rez = linprog(c_optim, A_ub=A_ub, b_ub=b_ub,
29                 A_eq=A_eq, b_eq=b_eq,
30                 bounds=bounds, method='highs')
31
32    print(f"Status: {rez.message}")
33    if rez.success:
34        for var, val in zip(nume_variabile, rez.x):
35            print(f" {var} = {val:.4f}")
36        val_obiectiv = -rez.fun if tip == 'max' else rez.fun
37        print(f" Valoare obiectiv ({tip}): {val_obiectiv:.4f}")
38    return rez
39
40 # --- Exemple de utilizare ---
41

```

```

42 print("=== DIETA ===")
43 rezolva_pl(
44     c=[1.0, 1.5, 2.0],
45     A_ub=[[-200, -150, -100], [-4, -8, -12], [-20, -200, -30]],
46     b_ub=[-1800, -60, -800],
47     tip='min',
48     nume_variabibile=['Paine', 'Lapte', 'Oua']
49 )
50
51 print("\n=== SILOZ ===")
52 rezolva_pl(
53     c=[120, 90, 80],
54     A_ub=[[1, 1, 1], [1, 0, 0], [0, 1, 0], [0, 0, 1], [-1, -1, 0]],
55     b_ub=[1000, 500, 600, 400, -400],
56     tip='max',
57     nume_variabibile=['Grau', 'Porumb', 'Orz']
58 )

```

8 Exerciții propuse

1. Formulați matematic și rezolvați cu `linprog` problema dietei extinse: adăugați un al patrulea aliment — **Carne** (350 kcal, 25g proteine, 10mg calciu, cost 5 lei/porție). Cum se modifică soluția optimă?
2. **Problema de producție:** o fabrică produce două produse P_1 și P_2 . Resursele disponibile sunt: 240 ore muncă, 120 kg materie primă. P_1 necesită 4 ore și 2 kg/buc, profitul este 5 lei/buc. P_2 necesită 2 ore și 4 kg/buc, profitul este 3 lei/buc. Formulați și rezolvați problema PL. Identificați grafic vârfurile regiunii fezabile și evaluați funcția obiectiv în fiecare.
3. Modificați problema transportului: dacă costul $S_1 \rightarrow D_3$ crește la 6 lei/unitate (în loc de 1), cum se modifică planul optim și costul total?
4. **Problema silozului extinsă:** administratorul poate închiria capacitate suplimentară la un cost de 10 lei/tonă (maxim 200 tone extra). Adăugați variabila de decizie x_4 (tone închiriate) și reformulați problema.
5. Verificați numeric că problema PL:

$$\max x_1 + x_2 \quad \text{s.t.} \quad x_1 - x_2 \leq 1, \quad -x_1 + x_2 \leq 1, \quad x_1, x_2 \geq 0$$

este **nemărginită**. Ce returnează `linprog` în acest caz? (*Indiciu:* verificați `rezultat.status`.)